Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 13.11.2023.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2024_Test1_GrupaE_IT1g2023**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Binarno pretraživanje*

Napisati ***sort*** metodu klase ***BinarySearch*** .

Dodatne napomene za rešavanje zadatka:

- Ulaz je sortiran niz celih brojeva
- Izlaz je true ili false, u zavisnosti da li je broj ppronađen
- U template-u se nalazi klasa ***SortedArrayGenerator*** koju treba iskoristiti za generisanje sortiranog niza slučajnih brojeva
- Kreirati klasu ***BinarySearch*** i u okviru nje generisati metodu ***public static bool search(long[] arr, int size, long value)***
- Testirati u okviru ***Main*** metode

Zadatak 2 – *Brisanje elemenata reda deljivih sa m, suma preostalih*

Implementirati metodu koja prima niz celih brojeva i jedan ceo broj ***m***. Metoda treba da koristi strukturu podataka red (klasa ***Queue***) za organizaciju i manipulaciju elementima, i treba da izvrši sledeće korake:

1. **Uklanjanje brojeva deljivih sa m**: Prolazom kroz red, ukloniti sve brojeve koji su deljivi sa zadatim brojem m.
2. **Računanje sume preostalih brojeva**: Izračunati zbir svih brojeva koji su ostali u redu nakon uklanjanja brojeva deljivih sa m.
3. **Vraćanje rezultata**: Kao rezultat, metoda treba da vrati izračunatu sumu preostalih brojeva u redu i ispisati te brojeve.

Dodatni zahtevi za rešavanje zadatka:

- Zadatak rešavati pomoću strukture ***reda*** (klasa ***Queue***) koja je data u okviru šablona zadatka.
- Nakon što se uklone brojevi deljivi sa m, red treba da ostane u istom redosledu kao pre uklanjanja tih brojeva. To znači da svi preostali brojevi (oni koji nisu deljivi sa m) treba da budu vraćeni u red u originalnom redosledu.
- Zadatak rešavati u okviru metode ***public int RemoveDivisibleAndSum(int[] array, int divisor)***
- Algoritam je neophodno napisati sa linearnom vremenskom kompleksnošću O(n).

Primer:

- Input: [3, 5, 7, 8, 10, 13, 14], m=3.
- Output: Suma elemenata je 57.
  Preostali elementi reda: 5, 7, 8, 10, 13, 14

- Input: [12, 18, 25, 40, 9, 30, 45], m=5
- Output: Suma elemenata je 39
  Preostali elementi reda: 12, 18, 9

Zadatak 3 – *Minimalna i maksimalna udaljenost kritičnih tačaka*

Kritična tačka u povezanoj listi se definiše kao lokalni maksimum ili lokalni minimum.

- Čvor je **lokalni maksimum** ako je vrednost trenutnog čvora strogo veća od vrednosti prethodnog i sledećeg čvora.
- Čvor je **lokalni minimum** ako je vrednost trenutnog čvora strogo manja od vrednosti prethodnog i sledećeg čvora.

Napomena: Čvor može biti lokalni maksimum ili minimum samo ako postoji prethodni i sledeći čvor.

Zadatak je da, s obzirom na početni čvor povezane liste (head), vratimo niz dužine 2 koji sadrži:

- **minDistance**: minimalnu udaljenost između bilo koja dva različita kritična čvora
- **maxDistance**: maksimalnu udaljenost između bilo koja dva različita kritična čvora

Ako postoji manje od dva kritična čvora, treba da se vrati niz [-1, -1].

Dodatni zahtevi za rešavanje zadatka:

- Zadatak rešavati pomoću strukture ***liste*** (klasa ***LinkedList***) koja je data u okviru šablona zadatka.
- Ne koristiti dodatne kolekcije poput liste ili niza, koristiti samo povezanu listu.
- Zadatak rešavati u okviru metode ***public int[] findMinMaxDistance()***

Primer:

- Input 1: 3 -> 1
- Output 1: [-1,-1]

- Input 2: 5 -> 3 -> 1 -> 2 -> 5 -> 1 -> 2
- Output 2: [1,3]

Objašnjenje:

- **Treći čvor** je **lokalni minimum** jer je vrednost 1 manja od 3 i 2.
- **Peti čvor** je **lokalni maksimum** jer je vrednost 5 veća od 2 i 1.

- **Šesti čvor** je **lokalni minimum** jer je vrednost 1 manja od 5 i 2.

Minimalna udaljenost između kritičnih tačaka je između petog i šestog čvora, što je udaljenost **1** (6 - 5 = 1).

Maksimalna udaljenost između kritičnih tačaka je između trećeg i šestog čvora, što je udaljenost **3** (6 - 3 = 3).